OPMIG-99: Best Practice Summary

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 10 of 10 | **Created:** March 2026 | **Last Updated:** 05/06/2026

Definitive best practice settings for migrating from classic logs to OpenPipeline. Each entry specifies the exact configuration.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail and OpenPipeline access
Knowledge	Completion of or familiarity with OPMIG-01 through OPMIG-09
Purpose	This notebook serves as a reference card — no active environment required

1. Migration Planning

Practice	Recommended Setting/Value	Priority
Inventory all log sources first	`fetch logs, from:-7d | summarize count(), by: {log.source} | sort count desc`	Critical
Score sources with weighted priority matrix	Volume 25%, Cost Savings 25%, Security Risk 25%, Parsing Complexity 10%, Business Criticality 15%	Critical
Execute migration in 4 waves	Wave 1 (weeks 1-2): critical/security. Wave 2 (3-4): high-volume. Wave 3 (5-6): standard prod. Wave 4 (7+): dev/test	Critical
Keep same API endpoint	`/api/v2/logs/ingest` works identically for Classic and OpenPipeline — no code changes	Recommended
Maintain `logs.ingest` token scope	Add `metrics.ingest` only if using metric extraction	Critical
Backup pipeline config before every change	`GET /api/v2/openpipeline/logs/pipelines/{id}` — store timestamped JSON backups	Critical

2. Pipeline Configuration

Practice	Recommended Setting/Value	Priority
One pipeline per use case	Name as `{source}-{purpose}` (e.g., `nginx-access-logs`)	Critical
Max 30 pipelines per scope	Consolidate with conditional processors if approaching limit	Recommended
Max 50 processors per pipeline	Split into multiple pipelines if exceeded	Recommended
Max 10 DQL commands per processor	Break into multiple processors if exceeded	Recommended
Processor order	Masking → Drop → Parse → Enrich → Transform	Critical
Name processors descriptively	`{action}-{target}` format (e.g., `mask-credit-cards`, `drop-debug-logs`)	Recommended
Test every processor with sample data	Use OpenPipeline UI "Sample data" field before saving	Critical
Configure default pipeline for unmatched data	Minimal processing, monitor volume — high counts = missing routing rules	Recommended
Max 5 pipelines per record	After 5, extraction stops but record still persists	Recommended

3. Routing Rules

Practice	Recommended Setting/Value	Priority
Specific routes before general routes	First match wins — compliance routes at highest priority	Critical
Max 100 routes per scope	Combine conditions with AND/OR to stay within limit	Recommended
Max 10 conditions per route	Use broader patterns if exceeded	Recommended
Route by `log.source` for source-based routing	`log.source == "nginx"` for specific apps	Recommended
Route by `k8s.namespace.name` for environments	`prod` → 35-day bucket, `staging` → 14-day, `dev` → 7-day	Recommended
Do NOT use `dt.entity.*` in routing conditions	Entity fields added AFTER Processing stage — not available during routing	Critical
Monitor unrouted log volume	`filter isNull(dt.openpipeline.pipelines)` grouped by `log.source` — must be 0	Critical
Storage uses first matching pipeline's bucket	Order routing so highest-retention pipeline matches first for compliance data	Critical

4. Bucket Strategy & Cost

Practice	Recommended Setting/Value	Priority
3-tier bucket strategy (small/medium orgs)	`critical_logs` 90d (10-15%), `default_logs` 35d (60-70%), `ephemeral_logs` 7d (20-30%)	Critical
5-tier bucket strategy (enterprise)	Add `compliance_logs` 365d (3-5%) and `security_logs` 180d (2-5%)	Critical
Bucket naming	`<environment>_<purpose>_logs` — max 100 chars, alphanumeric + underscore	Recommended
Drop DEBUG/TRACE before storage	Drop processor: `loglevel == "DEBUG" OR loglevel == "TRACE"` — 30-70% volume reduction	Critical
Drop health check logs	`contains(content, "/health") OR contains(content, "/ready") OR contains(content, "/metrics")` — 5-20% additional reduction	Recommended
Extract metrics then drop raw logs	1M requests/day as logs = ~$140/mo; as metrics = ~$1/mo (99.3% savings)	Recommended
Staging/dev short retention	Staging: 14 days (60% savings). Dev: 7 days (80% savings)	Recommended

Practice	Recommended Setting/Value	Priority
Review bucket strategy quarterly	Usage trends, retention validation, unused buckets, compliance audit	Recommended

5. Processing & Parsing

Practice	Recommended Setting/Value	Priority
Use built-in Technology Parsers first	JSON parser for JSON logs, Apache parser for access logs, Syslog for RFC 3164/5424	Recommended
Apache/Nginx DPL pattern	`IPADDR:client_ip SPACE '-' SPACE LD:user SPACE '[' TIMESTAMP(...):timestamp ']' SPACE '"' LD:method SPACE LD:request_path SPACE LD:protocol '"' SPACE INT:status_code SPACE INT:response_bytes`	Recommended
Java/Log4j DPL pattern	`TIMESTAMP('yyyy-MM-dd HH:mm:ss,SSS'):log_timestamp SPACE '[' LD:thread ']' SPACE LD:level SPACE LD:logger SPACE '-' SPACE DATA:message`	Recommended
Use alternatives for flexible extraction	`('user='|'userId='|'user_id=') LD:user_id` matches all variations	Recommended
Grok-to-DPL conversion	`%{IP}` → `IPADDR`, `%{INT}` → `INT`, `%{WORD}` → `WORD`, `%{GREEDYDATA}` → `DATA`	Recommended
Test patterns in DPL Architect	`https://{env}.apps.dynatrace.com/ui/apps/dynatrace.dpl.architect`	Critical
Target >90% parsing success rate	Query `countIf(isNotNull(loglevel))/count()` per pipeline — investigate below 90%	Recommended
Normalize missing log levels	`if(contains(content, "[ERROR]"), "ERROR", else: ...)` for logs without `loglevel`	Recommended

6. Metric Extraction (RED)

Practice	Recommended Setting/Value	Priority
Rate metric	Counter: `log.api.request_rate` with dimensions `method`, `path`, `status_category`, `service.name`	Recommended

Practice	Recommended Setting/Value	Priority
Error metric	Counter: `log.api.request_errors` matching `status >= 500 OR loglevel == "ERROR"`	Recommended
Duration metric	Value: `log.api.response_time_ms` with dimensions `service.name`, `endpoint`, `status_category`	Recommended
Metric naming convention	`<source>.<domain>.<metric>.<unit>` (e.g., `log.api.response_time_ms`)	Recommended
Max 5-7 dimensions per metric	Never use `user_id`, `request_id`, `session_id`, `client_ip` as dimensions	Critical
Keep cardinality below 10,000 series	Bucket high-cardinality fields (e.g., duration → fast/normal/slow)	Critical
Normalize API paths in dimensions	`/api/users/12345` → `/api/users/{id}`	Recommended
Max 10 metric extractions per pipeline	Max 5 event extractions, max 3 business event extractions	Recommended

7. Security & Masking

Practice	Recommended Setting/Value	Priority
Masking processors FIRST in every pipeline	Sensitive data redacted before parsing, routing, storage	Critical
Credit cards	Built-in `CREDITCARD` matcher → `[CC_REDACTED]`	Critical
CVV codes	`('cvv='|'cvc=') INT{3,4}` → `cvv=[REDACTED]`	Critical
Email addresses	Built-in `EMAIL` matcher → `[EMAIL_REDACTED]`	Critical
IP addresses (GDPR)	Built-in `IPADDR` matcher → `[IP_REDACTED]`	Critical
SSNs	`INT{3} '-' INT{2} '-' INT{4}` → `[SSN_REDACTED]`	Critical
Bearer tokens	`'Bearer ' NSPACE` → `Bearer [TOKEN_REDACTED]`	Critical
API keys	`('api_key='|'apiKey=') LD:key` → `api_key=[REDACTED]`	Critical
Passwords	`('password='|'pwd='|'passwd=') LD:pwd` → `password=[REDACTED]`	Critical
Remove always-sensitive fields	`fieldsRemove password, secret, api_key, token, authorization`	Recommended
Use descriptive placeholders	`[CC_REDACTED]`, `[EMAIL_REDACTED]`, `[PHI_REDACTED]` — enables audit counts by type	Recommended
Chain all masking in single processor	Apply CC, CVV, email, IP, SSN, tokens in sequence — single pass	Recommended

Practice	Recommended Setting/Value	Priority
Validate masking weekly	`filter matchesPhrase(content, "4111") OR matchesPhrase(content, "@gmail.com")` must return 0	Critical
Handle masking failure as critical incident	Disable pipeline, identify exposure window, revoke access, contact support	Critical

8. Compliance

Practice	Recommended Setting/Value	Priority
PCI-DSS	Bucket `pci_payment_logs`, 365 days, mask PANs/CVV, payment team + security + auditors only	Critical
SOC 2	Bucket `soc2_audit_logs`, 365 days, immutable after 30-day grace, security + auditors only	Critical
HIPAA	Bucket `hipaa_phi_logs`, 2555 days (7 years), mask patient IDs/SSNs/DOB/MRNs	Critical
GDPR	Bucket `eu_customer_logs`, 90 days max, mask emails/IPs, support erasure requests	Critical
Compliance routes at highest priority	Regulatory data must never accidentally route to under-retained buckets	Critical
Annual compliance audit	Verify retention, masking, erasure procedures, encryption, access controls	Recommended

9. Troubleshooting & Validation

Practice	Recommended Setting/Value	Priority
Post-migration validation checklist	All sources flowing, volumes match, no data loss, timestamps correct, routing correct, masking working, metrics generating	Critical
Monitor pipeline volume hourly	`makeTimeseries count(), by:{dt.openpipeline.pipelines}, interval:1h` — drops = routing failures	Recommended
Check log source health	`summarize last_seen = max(timestamp), by:{log.source}` — sources silent >30 min need investigation	Recommended
Combine regex for performance	Single alternation `(p1|p2|p3)` instead of chained `replaceAll` — 50-80% faster	Recommended
Drop processors before parse processors	Reduces volume before expensive parsing — 60-90% reduction for verbose apps	Critical

Practice	Recommended Setting/Value	Priority
Never drop ERROR/FATAL in production	Safeguard: `loglevel == "DEBUG" AND NOT (loglevel == "ERROR" OR loglevel == "FATAL")`	Critical
Test changes in staging first	Backup → deploy staging → validate → document → rollback plan → production	Critical

10. Data Limits & Constraints

Practice	Recommended Setting/Value	Priority
Max record size	1 MB before processing, 16 MB after — exceeding = rejected/dropped	Critical
Timestamp window	Logs: 24h past to 10min future. Spans: 2h past. Metrics: 1h past to 1min future	Critical
Max fields per record	1,000 fields, 10 levels JSON nesting, 32 KB per attribute value, 1,000 array elements	Recommended
Rate limits	Log Ingest API: 500 req/min/token. OTLP: 1,000 req/min. Config API: 100 req/min	Recommended

---

11. DQL Cookbook

Reference queries consolidated from across the OPMIG series — copy / paste into your tenant for ad-hoc validation, monitoring, or troubleshooting. These were previously scattered through OPMIG-09 (Troubleshooting & Validation); they live here as a cookbook so the main notebook stays focused on teaching content.

11.1 Parsing Validation

Diagnostic queries to check whether your DQL/DPL parsers are working correctly. Run these after configuring or modifying parse processors.

```
// Parsing success rate by pipeline
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    with_loglevel = countIf(isNotNull(loglevel)),
    without_loglevel = countIf(isNull(loglevel))
  }, by: {dt.openpipeline.pipelines}
| fieldsAdd parse_rate = round((toDouble(with_loglevel) / toDouble(total)) * 100, decimals: 1)
| sort parse_rate asc
```

```
// Log level distribution (verify expected values)
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {loglevel}
| sort log_count desc
```

```
// Find log sources with low parsing rates
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    parsed = countIf(isNotNull(loglevel))
  }, by: {log.source}
| fieldsAdd parse_rate = round((toDouble(parsed) / toDouble(total)) * 100, decimals: 1)
| filter parse_rate &lt; 90
| sort parse_rate asc
```

```
// Sample unparsed logs from problematic sources
// Replace 'your-source' with source from previous query
fetch logs, from: now() - 1h
| filter log.source == "your-source"
| filter isNull(loglevel)
| fields content
| limit 30
```

```
// Verify custom field extraction
// Add your expected custom fields here
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    with_request_id = countIf(isNotNull(request_id)),
    with_user_id = countIf(isNotNull(user_id)),
    with_order_id = countIf(isNotNull(order_id))
  }, by: {dt.openpipeline.pipelines}
```

11.2 Volume & Cost Validation

Verify post-migration data volumes match expectations and that drop processors are reducing volume as designed. Useful in the first week after cutover and as a recurring check.

```
// Daily log volume by bucket (cost impact)
fetch logs, from: now() - 7d
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {log_count = count()}, by: {day, dt.system.bucket}
| sort day desc, log_count desc
```

```
// Volume trend - verify stable after migration
fetch logs, from: now() - 7d
| makeTimeseries {log_count = count()}, interval: 1h
```

```
// Check if debug/trace logs are being dropped (cost savings)
fetch logs, from: now() - 24h
| filter loglevel == "DEBUG" OR loglevel == "TRACE"
| summarize {debug_trace_count = count()}
| fieldsAdd status = if(debug_trace_count == 0,
    "✅ Debug/trace logs dropped as expected",
    else: "⚠️ Debug/trace logs still present - check drop processors")
```

```
// Volume by log level (should see mostly INFO/WARN/ERROR)
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {loglevel}
| fieldsAdd percentage = round((toDouble(log_count) / 100), decimals: 1)
| sort log_count desc
```

```
// Top 10 highest volume sources
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {log.source}
| sort log_count desc
| limit 10
```

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.